

QVC-Scale Broadcast — one feed, a huge audience

How a single live-shopping feed carries a QVC-sized crowd: WebRTC for interaction, HLS over a CDN for the masses.

The short answer to “can enough people watch one feed?”: yes. The trick is not to make the SFU serve everyone — it’s to use the right tool for each part of the audience. An SFU (LiveKit) forwards a private copy to every viewer, which is perfect for interaction but caps out around ~1,000 viewers per node and gets expensive fast. A **CDN**, by contrast, serves the same video segments to *millions* of people cheaply, because the edge caches them. So the platform now runs a **hybrid**: the host (and any interactive guests) stay on low-latency WebRTC, and the mass passive audience watches an **HLS stream delivered by your CDN**. Passive viewers never touch the SFU, so one feed scales as far as your CDN does.

The two tiers of one feed

	Interactive tier (WebRTC/SFU)	Broadcast tier (HLS/CDN)
Who	Host + a small interactive group	The mass audience (watch & buy)
Latency	Sub-second	~2-6s (LL-HLS ~2-4s)
Scales to	~1,000 per SFU node	Essentially unlimited (CDN-bound)
Cost driver	SFU nodes (autoscaled, capped)	CDN egress (cheap, cacheable)
Cap	HOSTING_MAX_VIEWERS_PER_ROOM (default 50)	None — the CDN carries it

A feed starts on the interactive tier. When it goes **broadcast** — the host taps *Go broadcast*, or the room auto-switches once it reaches `HOSTING_AUTO_BROADCAST_THRESHOLD` (default 200) — LiveKit Egress composites the room into an HLS stream, and every new passive viewer is handed that CDN URL instead of an SFU subscription. Buying still works: the featured product is mirrored to `sessionFeatured` (which HLS viewers poll), and *Buy now* goes through the same `liveShoppingOrder` path in Site Cash.

So the “50 viewers per room” number you asked about only ever bounds the **interactive** tier. The broadcast tier has no such cap — that’s the QVC-scale path.

What it costs at QVC scale

Passive viewers cost **CDN egress**, not SFU nodes. The SFU only ever carries the host + interactive group (a handful of nodes at most), no matter how big the crowd. Two things make the crowd cheap:

- **The SFU cost stays flat.** 100,000 passive viewers = **0 extra SFU**

nodes (they're all on HLS). Egress produces *one* stream; the CDN fans it out.

- **The CDN can be near-zero-egress.** The cheapest path is **Cloudflare R2** (object storage with **zero egress fees**) fronted by **Cloudflare's CDN** — you pay for storage and requests, but not for the bandwidth out to viewers. On S3 + CloudFront you'd pay CDN egress (~\$0.02–0.08/GB), still far cheaper and more scalable than SFU forwarding.

At an 800 kbps stream (~0.36 GB/viewer-hour): 100,000 concurrent viewers for one hour is ~36 TB of delivery. On Cloudflare R2→CDN that's dominated by storage/request costs (small); on a metered CDN at ~\$0.02/GB it's on the order of ~\$700 for that hour — versus needing ~100 SFU nodes to do the same thing on WebRTC. The broadcast path is what makes a QVC-sized audience affordable at all.

How to turn it on

It's **on by default in code** (`HOSTING_BROADCAST_ENABLED = 1`) and safe until the media pieces are configured. To actually produce streams you deploy one more component and set two URLs:

1. **Deploy LiveKit Egress** (the compositor that writes HLS). On Railway, add it as a second service; on a VPS, run the egress container. It uses the same `LIVEKIT_API_KEY/SECRET`.
2. **Point it at storage → CDN.** Configure Egress's storage to your bucket (**Cloudflare R2 recommended**), and put your CDN in front. Set:
 - `LIVEKIT_EGRESS_URL` = your Egress service URL.
 - `HLS_PLAYBACK_BASE_URL` = your CDN base (e.g. `https://cdn.yourdomain.com/live`). The room's playlist is `<base>/<room>/index.m3u8`.
 - `HLS_STORAGE_BUCKET` = the bucket Egress writes to (credentials stay in Egress's own config / server env — never in the client).
3. That's it. `sessionBroadcastStart` mints an egress token, starts the HLS egress, and records the CDN URL on the session; the token endpoint then serves passive viewers HLS automatically.

Until those are set, `sessionBroadcastStart` returns `configured:false` (a safe no-op) and viewers stay on the interactive tier. Live hosting itself remains counsel-gated (`SESSION_HOSTING_ENABLED` off) — this only makes the broadcast path ready.

Scaling the metadata, not just the video (poll → CDN)

The video already scales — one HLS feed off the CDN serves an unlimited crowd. The one thing that *didn't* scale with it was the little **metadata poll**: HLS viewers aren't in the WebRTC room, so they can't receive the host's "feature this product" / "ad break now" data ping. They poll `sessionFeatured` for it. Naively, 100k viewers polling an origin function every few seconds is 100k×/tick of origin load — the crowd moved off the SFU only to pile onto a function.

Two changes fix that so the poll scales with the video:

1. **The origin poll is now short-cache + SWR.** `sessionFeatured` (GET) returns `cache-control: public, max-age=3, stale-while-revalidate=15`. Any CDN/edge in front of the origin collapses a viewer burst into ~one origin hit every few seconds instead of one per viewer — the state only changes when the host advances a product, so a 3s cache is invisible to viewers.

2. **Viewers prefer a static CDN state file.** When broadcast is configured, the room's state (featured product + ad-break marker) belongs next to the HLS segments as `<base>/<room>/state.json` (`stateUrlForRoom`). The player learns this `state_url` from its first origin poll, then polls the **CDN object** — edge-served, essentially free, and it scales exactly like the video does. If the object isn't published yet, the client falls back to the (now-cacheable) origin poll, so it's always safe.

The client poll interval also widened from 4s to 8s — featured-product and ad-break changes are human-paced, so 8s is plenty and halves the request rate. Publishing `state.json` to the bucket (from the host/egress side) is the switch that lights the CDN path fully; until then the SWR-cached origin already removes the per-viewer origin cost.

The one honest tradeoff

The broadcast audience is **not** sub-second live — HLS adds ~2-6 seconds (low-latency HLS ~2-4s). For watch-and-buy that's exactly right and is what every large live-shopping platform does. Anyone who needs true real-time interaction (the host, co-hosts, a small VIP group) stays on the WebRTC tier and keeps sub-second latency. You get both at once, on the same feed.

Verified by the load test (`deploy-kit/load-test.mjs`): broadcast routing switches the crowd to HLS at the threshold, and a 100k-viewer feed uses zero SFU nodes for the passive audience. See `LIVEKIT-HOSTING-COST-MODEL.md` and `COST-FLOOR-AND-LLAMA-ROUTING.md`.